



D-Bus

The Smart, Simple, Powerful IPC

Let's learn the intricacies of D-Bus and use it to hack some nifty features into programs.

Inter-process Communication (IPC) helps applications to talk to each other. You might have seen Firefox automatically tuned to offline mode when your Internet connection is down. Ever wondered how this happens? This is because the NetworkManager application talks to Firefox using a back-end utility called D-Bus to update it on the status of the Internet connection.

D-Bus (Desktop Bus) is a simple IPC, developed as part of freedesktop.org project. It provides an abstraction layer over various applications to expose their functionalities and possibilities. If you want to utilise some feature of an application to make another program perform a specific task, you can easily implement it by making the process D-Bus aware. Once an application is made D-Bus compliant there's no need to recompile or embed code in it to make it communicate with other applications.

One thing really cool about D-Bus is that it helps developers write code for any D-Bus compliant application in a language of their choice. Currently, D-Bus bindings are available for C/C++, Glib, Java, Python, Perl, Ruby, etc.

Understanding D-Bus

D-Bus is a service daemon that runs in the background. We use bus daemons to interact with applications and their functionalities. The bus daemon forwards and receives messages to and from applications. There are two types of bus daemons: SessionBus and SystemBus.

The daemon that is attached to each user session is called SessionBus. When a user logs in, applications launched by him are attached to the SessionBus—a local bus limited to communicating between desktop applications that belong to a specific user currently logged in.

On the contrary, SystemBus is system-wide. It is initiated when the system boots, and is 'global' to the operating system. It is capable of interacting with the kernel and various system-wide events. Hardware Abstraction Layer (HAL), NetworkManager and udev are applications that use SystemBus.

In this article, I will use Python bindings to explore the D-Bus daemon. To begin with, if we want to use a desktop-level conversation, a SessionBus object can be created as follows:

```
[slynnu@slynnux-laptop dbus-python-0.83.0]$ python
Python 2.5.2 (r252:60911, Sep 30 2008, 15:41:38)
[GCC 4.3.2 20080917 (Red Hat 4.3.2-4)] on linux2
```